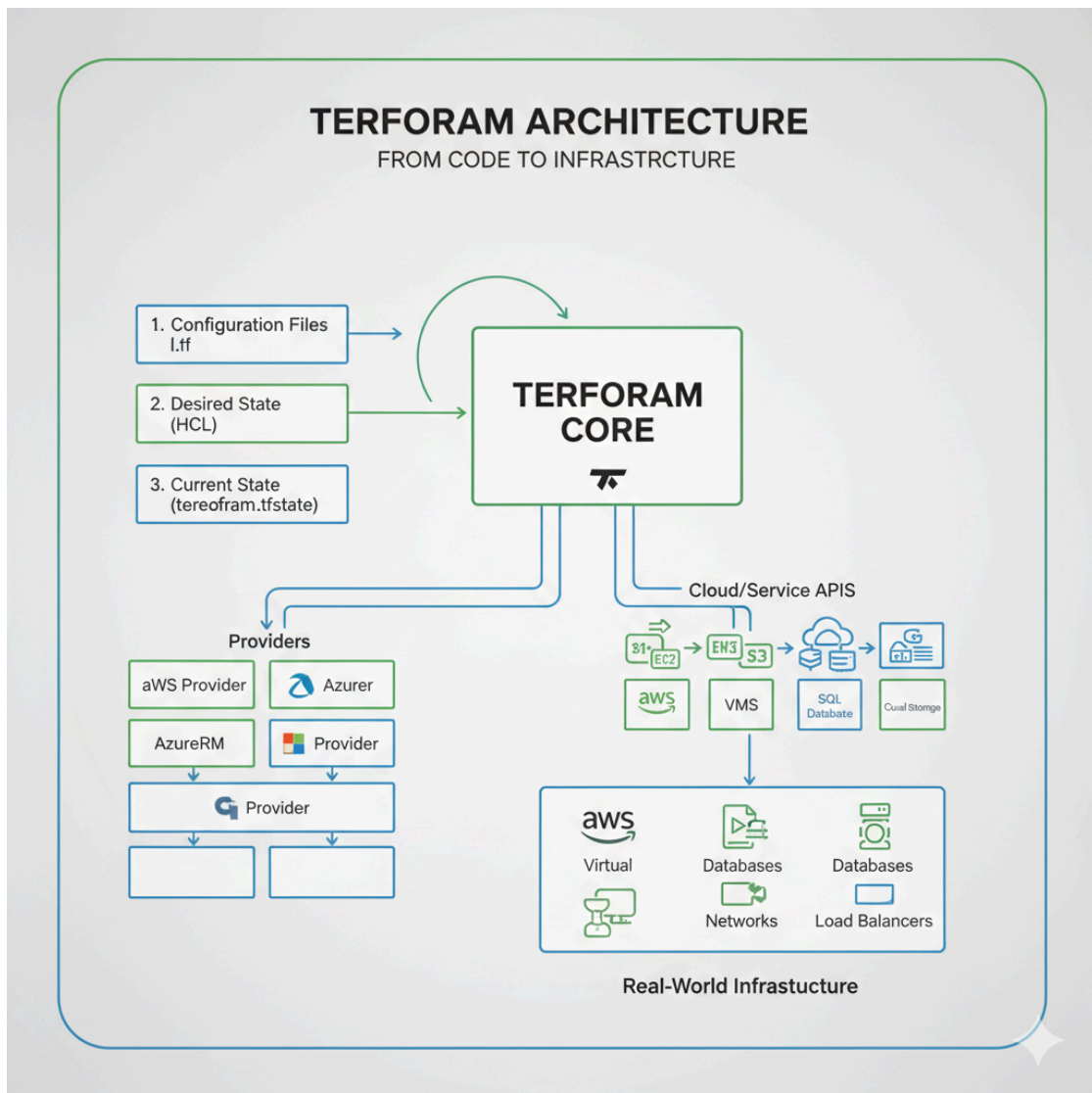


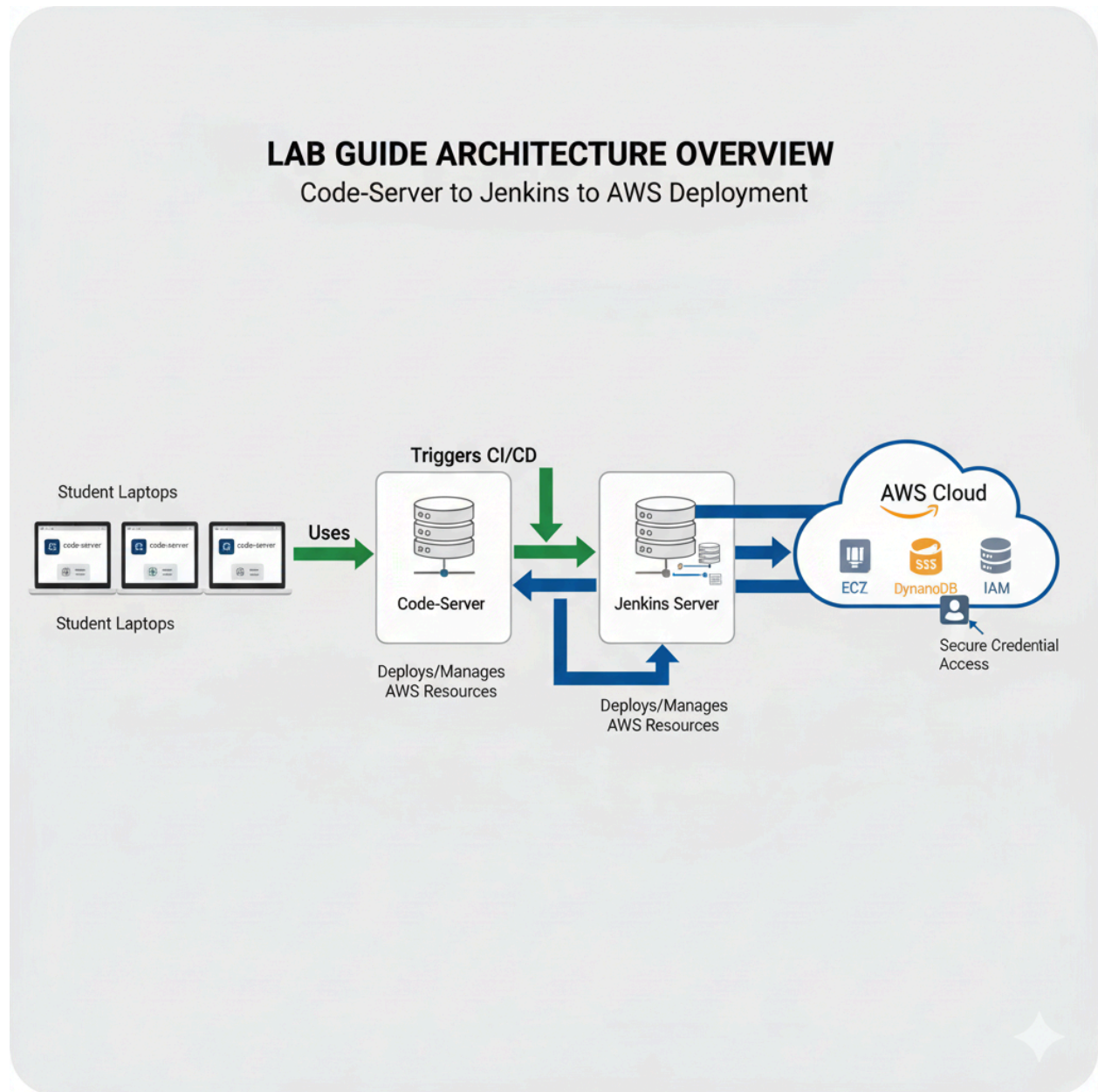
Terraform Fundamentals Reference Guide

What is Terraform?

Terraform is an **open-source Infrastructure as Code (IaC) tool** created by HashiCorp. It allows users to define and provision data center infrastructure using a declarative configuration language.



Our Custom lab environment :



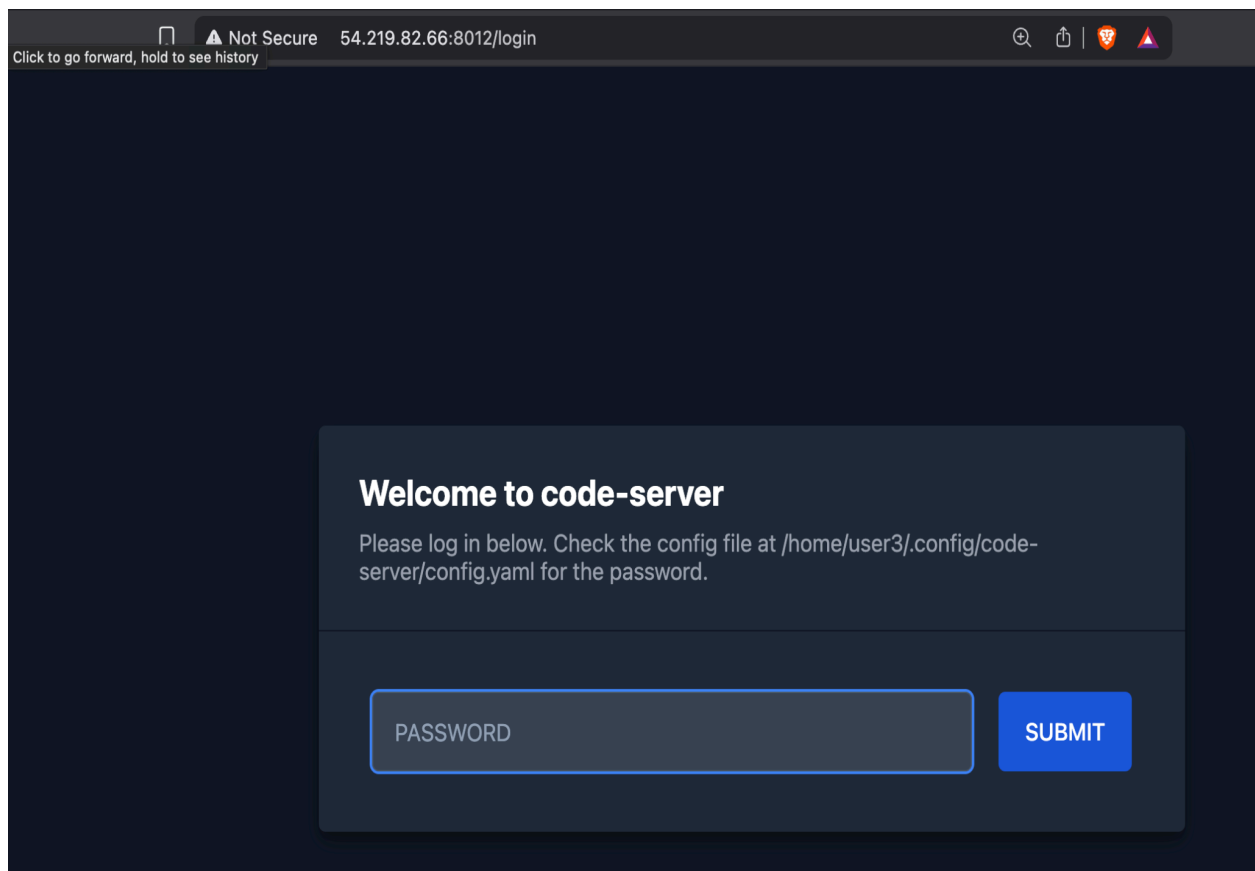
Code-server 1 → Login details (vscode-server)

Code-server 2 → Login details (vscode-server)

Note: Please refer xls file

Code-server Login Process 👍

Step 1: Login to URL



The screenshot shows a web browser window with the address bar displaying "54.219.82.66:8012/login". The page has a dark theme. A central box contains the text "Welcome to code-server" and a message: "Please log in below. Check the config file at /home/user3/.config/code-server/config.yaml for the password." Below this, there is a text input field labeled "PASSWORD" and a blue "SUBMIT" button.

Click to go forward, hold to see history

Not Secure 54.219.82.66:8012/login

Welcome to code-server

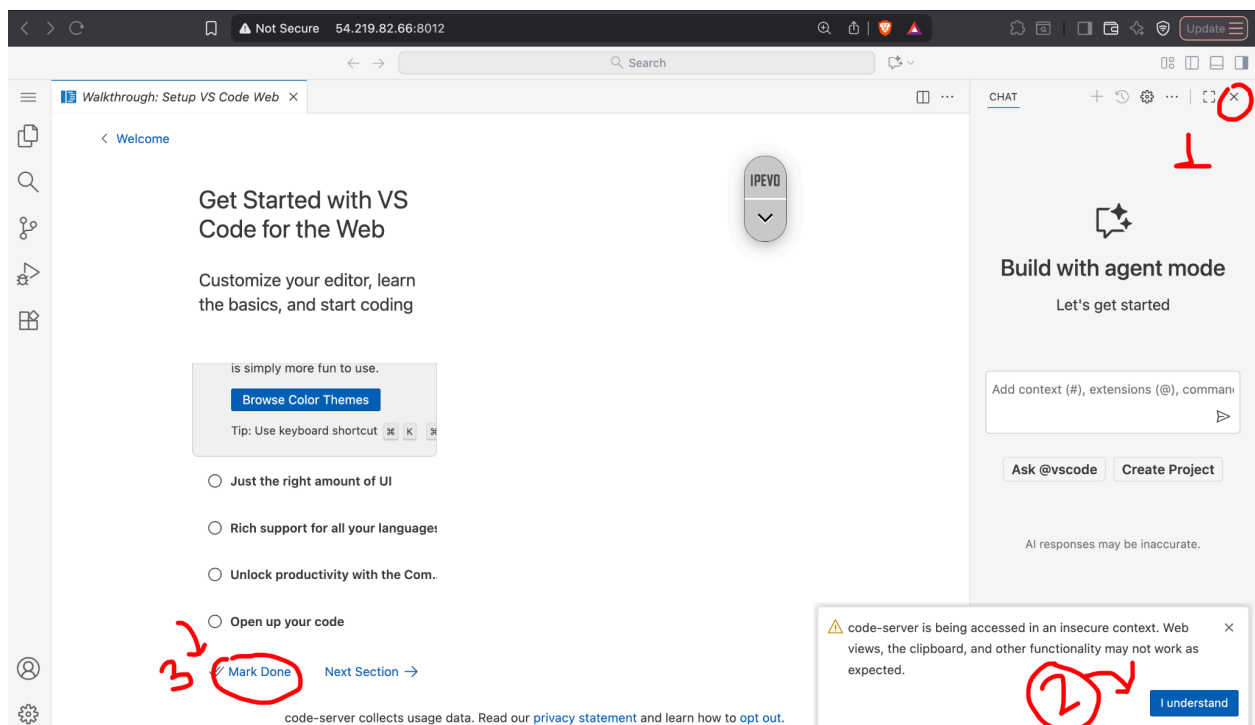
Please log in below. Check the config file at /home/user3/.config/code-server/config.yaml for the password.

PASSWORD

SUBMIT

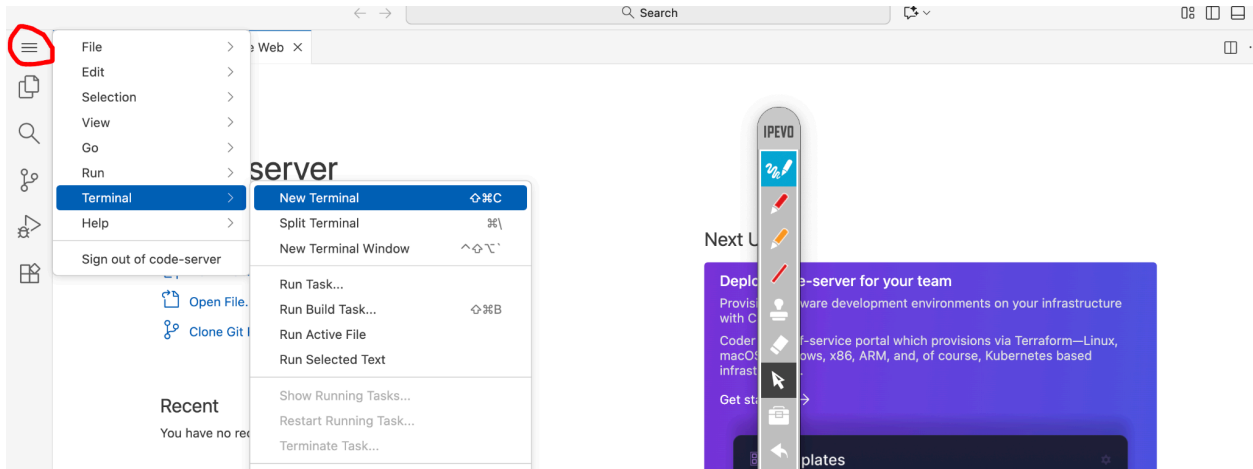
Step 2 : Enter password and you will get screen given below

Note: click on points 1 , 2 , 3 if visible else ignore



Step 3 : access terminal and run few commands

Click on left hand top corner in vscode





Walkthrough: Setup VS Code Web ×



Start



New File...



Open File...



Clone Git Repository...

Recent

You have no recent folders, [open a folder](#) to start.

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

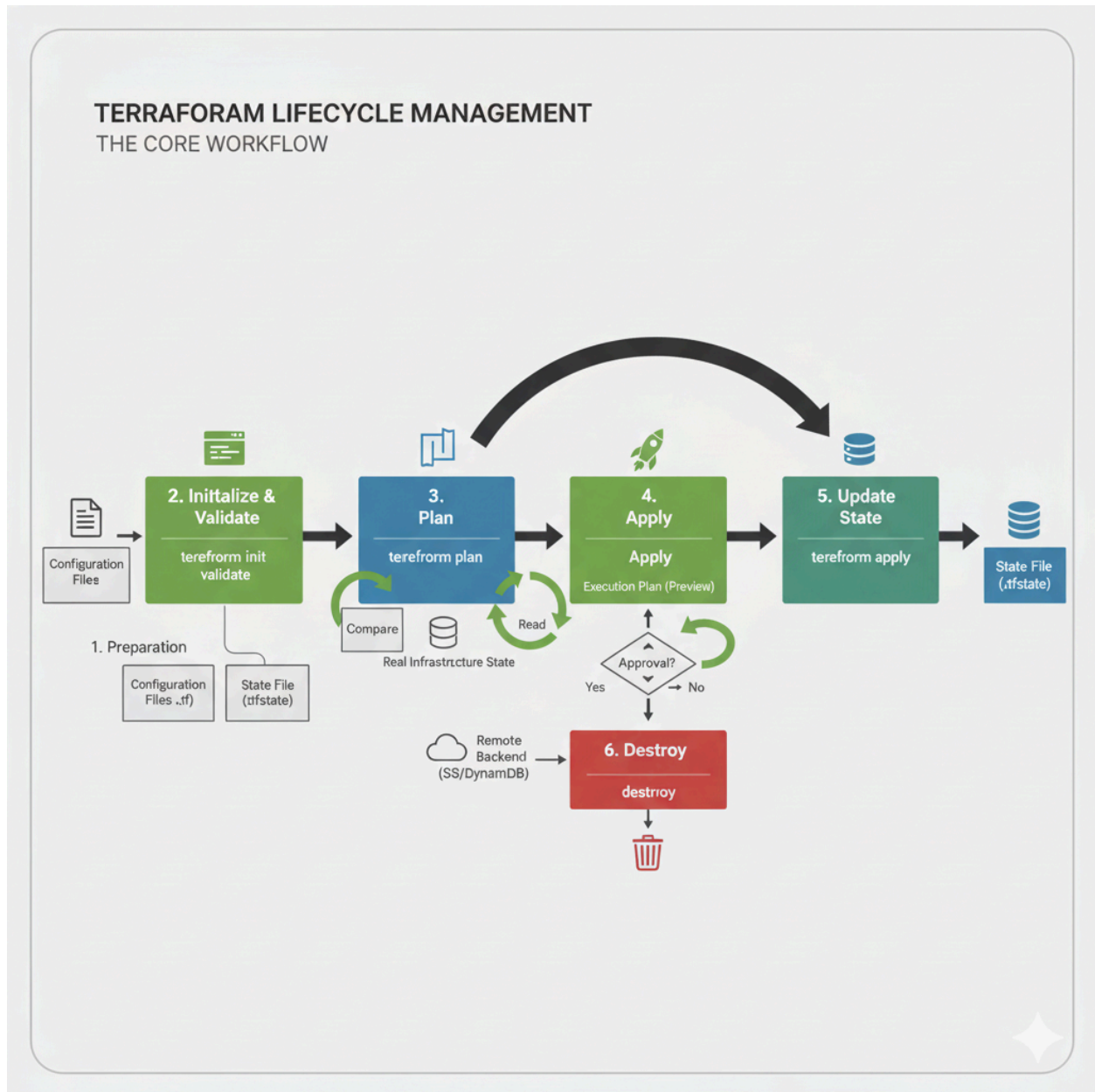
PORTS

- [user3@ip-172-31-26-112 ~]\$ terraform version
Terraform v1.13.5
on linux_amd64
- [user3@ip-172-31-26-112 ~]\$ git --version
git version 2.50.1
- [user3@ip-172-31-26-112 ~]\$ aws s3 ls
2025-03-03 15:06:51 delvex-software-center
- [user3@ip-172-31-26-112 ~]\$ █



Basic terraform commands for most of the cases

i) Terraform life cycle



Terraform commands for any resources to plan :

INIT 🖐️ (terraform init)

The `terraform init` command initializes a working directory by downloading necessary provider plugins and setting up the backend configuration. It is the first command you must run for any new or cloned Terraform project

```
[root@ip-172-31-26-112 terraform_B0A_test]# ls
README.md  a.txt  poc.tf
[root@ip-172-31-26-112 terraform_B0A_test]#
[root@ip-172-31-26-112 terraform_B0A_test]#
[root@ip-172-31-26-112 terraform_B0A_test]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/null...
- Finding hashicorp/aws versions matching ">= 4.0.0"...
- Installing hashicorp/null v3.2.4...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
- Installing hashicorp/aws v6.21.0...
- Installed hashicorp/aws v6.21.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

PLAN (terraform plan)

The `terraform plan` command compares the desired configuration with the actual infrastructure state to generate an execution plan, showing exactly what Terraform will create, update, or destroy before making any changes.

```
[root@ip-172-31-26-112 terraform_B0A_test]# terraform plan
data.aws_region.current: Reading...
data.aws_region.current: Read complete after 0s [id=us-east-1]

Terraform used the selected providers to generate the following execution
+ create

Terraform will perform the following actions:

# null_resource.hello will be created
+ resource "null_resource" "hello" {
  + id          = (known after apply)
  + triggers = {
    + "always_run" = (known after apply)
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

APPLY (terraform apply)

The `terraform apply` command executes the execution plan generated by `terraform plan`, provisioning the defined infrastructure and updating the state file to reflect the new resource status.

```
[root@ip-172-31-26-112 terraform_B0A_test]# terraform apply
data.aws_region.current: Reading...
data.aws_region.current: Read complete after 0s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource
+ create

Terraform will perform the following actions:

# null_resource.hello will be created
+ resource "null_resource" "hello" {
+   id          = (known after apply)
+   triggers = {
+     "always_run" = (known after apply)
+   }
+ }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ aws_region          = "us-east-1"
+ hello_resource_id = (known after apply)

Warning: Deprecated attribute

on poc.tf line 49, in output "aws_region":
49:   value          = data.aws_region.current.name

The attribute "name" is deprecated. Refer to the provider documentation for details.

(and one more similar warning elsewhere)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value:
```

Cheatsheet: Terraform commands

1 Initialization & Setup

Initialize a new Terraform working directory

```
terraform init
```

Output:

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching
"6.20.0"...
- Installing hashicorp/aws v6.20.0...
Terraform has been successfully initialized!
```

You may now begin working with Terraform.

Re-initialize and re-download providers

```
terraform init -upgrade
```

Output:

```
Upgrading provider plugins...
```

```
- Upgrading hashicorp/aws from v5.49.0 to  
v6.20.0...
```

```
Terraform has been successfully upgraded!
```

2 Planning Infrastructure

Show what changes will be made

```
terraform plan
```

Output:

```
Terraform will perform the following actions:
```

```
# aws_instance.my_ec2_instance[0] will be  
created  
+ resource "aws_instance" "my_ec2_instance" {  
    + ami           = "ami-03ff9720b16e8da76"  
    + instance_type = "t2.large"  
    + tags          = {  
        + "Name" = "jenkins-slave-1"
```

```
    }  
  }  
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Save plan to a file (for review or CI/CD)

```
terraform plan -out=tfplan
```

Output:

Plan: 1 to add, 0 to change, 0 to destroy.

Saved the plan to: tfplan

Show saved plan contents

```
terraform show tfplan
```

Output:

```
# aws_instance.my_ec2_instance[0] will be  
created  
+ resource "aws_instance" "my_ec2_instance" {  
    + ami              = "ami-03ff9720b16e8da76"  
    + instance_type    = "t2.large"  
}
```

③ Applying Infrastructure

Apply changes interactively

```
terraform apply
```

Output:

```
Terraform will perform the following actions:
```

```
# aws_instance.my_ec2_instance[0] will be created
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

```
Do you want to perform these actions?
```

```
Enter a value: yes
```

```
aws_instance.my_ec2_instance[0]: Creating...
```

```
aws_instance.my_ec2_instance[0]: Creation complete after 32s
```

```
[id=i-1234567890abcdef]
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Apply using saved plan (non-interactive)

```
terraform apply tfplan
```

Output:

```
aws_instance.my_ec2_instance[0]: Creation complete after 31s
```

```
[id=i-1234567890abcdef]
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Auto approve (use in automation only)

```
terraform apply -auto-approve
```

4 Destroying Infrastructure

Destroy resources

```
terraform destroy
```

Output:

```
Terraform will perform the following actions:
```

```
# aws_instance.my_ec2_instance[0] will be destroyed
```

```
Plan: 0 to add, 0 to change, 1 to destroy.
```

```
Do you really want to destroy all resources?
```

```
Enter a value: yes
```

```
aws_instance.my_ec2_instance[0]: Destroying...
```

```
[id=i-1234567890abcdef]
```

```
aws_instance.my_ec2_instance[0]: Destruction complete after  
15s
```

```
Destroy complete! Resources: 1 destroyed.
```

Auto approve destroy

```
terraform destroy -auto-approve
```

5 Inspecting State

List all resources in state

```
terraform state list
```

Output:

```
aws_instance.my_ec2_instance[0]  
aws_s3_bucket.example  
aws_iam_role.ec2_role
```

Show details of a resource

```
terraform state show aws_instance.my_ec2_instance[0]
```

Output:

```
# aws_instance.my_ec2_instance[0]:  
resource "aws_instance" "my_ec2_instance" {  
    ami           = "ami-03ff9720b16e8da76"  
    id            = "i-1234567890abcdef"  
    instance_type = "t2.large"  
    public_ip     = "54.190.10.34"  
}
```

Remove resource from state (without deleting in cloud)

```
terraform state rm aws_instance.my_ec2_instance[0]
```

Output:

```
Removed aws_instance.my_ec2_instance[0]
```

Move or rename resource in state

```
terraform state mv aws_instance.old_name  
aws_instance.new_name
```

Output:

```
Move "aws_instance.old_name" to "aws_instance.new_name"
```

6 Outputs & Variables

View all outputs

```
terraform output
```

Output:

```
public_ip = "54.190.10.34"  
instance_id = "i-1234567890abcdef"
```

Show single output

```
terraform output public_ip
```

Output:

```
54.190.10.34
```

Pass variables from CLI

```
terraform apply -var "instance_type=t3.medium" -auto-approve
```

Use variable file

```
terraform apply -var-file="dev.tfvars"
```

7 Debugging & Logs

Enable debug logs

```
export TF_LOG=DEBUG
terraform apply
```

Output (trimmed):

```
2025-11-13T10:21:23.456Z [DEBUG]
provider.terraform-provider-aws: ...
```

Turn off logs:

```
unset TF_LOG
```

Show current Terraform version

```
terraform version
```

Output:

```
Terraform v1.8.4  
on linux_amd64  
+ provider registry.terraform.io/hashicorp/aws v6.20.0
```



⑧ Formatting, Validation & Docs

Check syntax and validity

```
terraform validate
```

Output:

```
Success! The configuration is valid.
```

Format all Terraform files

```
terraform fmt
```

Output:

```
main.tf  
outputs.tf  
variables.tf
```

Generate documentation for variables

```
terraform providers
```

Output:

```
Providers required by configuration:
```

```
.  
└─ provider[registry.terraform.io/hashicorp/aws] ~> 6.20.0
```

9 Importing & Refreshing

Import existing resource into Terraform

```
terraform import aws_s3_bucket.example  
delvex-software-center
```

Output:

```
aws_s3_bucket.example: Importing from ID  
"delvex-software-center" ...  
Import successful!
```

Refresh state with live infrastructure

```
terraform refresh
```

Output:

```
aws_instance.my_ec2_instance[0]: Refreshing state...  
[id=i-1234567890abcdef]
```

10 Workspaces (Multi-Environment Support)

List workspaces

```
terraform workspace list
```

Output:

```
* default
```

dev
staging
prod

Create new workspace

`terraform workspace new staging`

Output:

Created and switched to workspace "staging"!

Switch workspace

`terraform workspace select prod`

Show current workspace

`terraform workspace show`

Output:

prod

Generate dependency graph

```
terraform graph | dot -Tpng > graph.png
```

Output:

```
Graph written to graph.png
```

Show resource dependency plan

```
terraform show
```

Output:

```
# aws_instance.my_ec2_instance:
resource "aws_instance" "my_ec2_instance" {
  ami = "ami-03ff9720b16e8da76"
  instance_type = "t2.large"
}
```



Clean-up and Maintenance

Remove cached plugins and init data

```
rm -rf .terraform  
rm .terraform.lock.hcl  
terraform init
```

Upgrade Terraform binary

```
terraform -install-autocomplete
```

Output:

```
Terraform command-line auto-completion activated  
for bash/zsh.
```